AGENT HW Energy Agent **RUNNING USER** (hypothetical grant model – permission set: HW_ServiceAgent)
CHANNEL agent **FINGERPRINT** da5d75d3171b **GENERATED** measured 2026-08-29

AGENT BLAST RADIUS REPORT

HW Energy Agent

Static, zero-credit analysis of the agent's real data-access surface at the execution-semantics layer.

AKSU INDEX

0

fields proven reachable beyond this user

No field escalation could be proven for this agent and this user. Proven is not the same as safe — read the two columns to the right.

0 of those carry the org's own **regulated** compliance label. This bucket is a subset of the number above, never an addition to it.

0

unproven boundaries

Real access boundaries the analysis could not prove were crossed. Reported separately and **never merged into proven** — severity is this tool's proof claim.

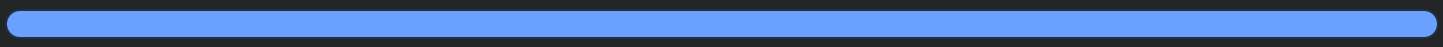
2

unresolved

Reach the analysis could not determine at all — dynamic SOQL, or a run context that stayed undetermined. An unknown never reads as clean.

0 proven with unresolved reach is NOT clean — unknown never becomes clean. 2 reach paths could not be resolved at all, so this report proves nothing about them. Read this as **unproven**, not as a pass.

WHAT THE AGENT REACHES VS. WHAT THE USER SEES – FIELDS



4 fields the agent's code reaches

4 of those, this user could read anyway

0 the gap between them (0 proven · 0 unproven)

Across **4** objects the agent's code touches. This report does not compute the set of objects the running user can see, so no object-level comparison is claimed here.

Aksu Index: 0 proven (0 regulated) · 0 unproven boundaries · 2 unresolved
One agent × one running user, at one moment, under one tool version — not an org score.

AKSU:1.0/P:0/C:0/B:0/U:2/fp:da5d75d3171b

https://aksuindex.com/spec/v1.0

NEEDS A LOOK

Nothing was proven to escape here — but part of this agent's reach could not be determined at all.

We statically resolved every action discovered behind **HW Energy Agent** — **9** action(s), reaching **4** data object(s) and **4** field(s) — and compared that against what its user is actually allowed to see. Reach that could not be resolved is reported separately as **unresolved**, never folded into this. Nothing was run and no data left the org.

On **records**: every *resolved* read the agent performs enforces sharing and the user's permissions, so on those reads the agent cannot see records this user could not see anyway. There is no *proven* record-level gap to fix.

Every field this scan could resolve carries a classification lookup (100% of resolved fields), so the personal-data result above is a real measurement rather than a blind spot — within what was resolved. Unresolved reach is counted separately.

*The exact fixes are listed under **What to do next** below.*

SECURITY POSTURE — BY APEX API VERSION

8
APEX CLASSES ANALYSED

8
at API v67+
secure-by-default (user mode)

0
pre-v67
system-mode default — sharing/FLS bypassed by default

Every analysed Apex class runs at API v67 or later, so the platform enforces the running user's access by default. This is the strong baseline; the findings still apply to explicit system-mode code, Flows and legacy triggers.

Evidence grade: every Apex action was read from a real parse tree, so the Authority Path is traced and a finding downgraded to WARN was *proven* not to reach the model — not merely unobserved.

No field escalation proven — but the reach is not fully resolved.

No field the agent's code reads is one its running user cannot see — among the reach we could resolve. **2** reach paths stayed undetermined, so this is "nothing proven", not "nothing there".

9	4	4	2/9	100%
ACTIONS	OBJECTS REACHABLE	FIELDS REACHABLE	SYSTEM-MODE	CLASSIFICATION COVERAGE

RECORD REACH — THE SYSTEM-MODE CEILING, NOT A MEASURED RESULT

> Every *resolved* read the agent performs **enforces sharing**, so the agent is bounded by the running user on those reads — there is **no proven record escalation**.

OBJECT	READ MODE	RECORDS IN ORG	USER SEES	GAP (UPPER BOUND)	CAUSE
`Case`	user	33	*= agent (sharing enforced)*	0	—
`Tariff_Change_Request__c`	user	1	*= agent (sharing enforced)*	0	—

> Only objects the agent's code actually **reads** are counted here (a create/insert target is a write, not a read of N records). **Records in org** is a live `COUNT()` of the whole object — it is an **upper bound, not the agent's result**: query predicates and `LIMIT` are not resolved statically. It is an escalation ceiling only for **system-mode** reads; a **user-mode** read enforces sharing, so the agent is bounded by the running user and the gap is 0 by construction. **n/a** marks record visibility that is sharing/ownership dependent and cannot be measured without running as the user — it is never estimated.

WHAT TO DO NEXT

Each item below is a concrete fix. Clear them and re-run; the report regenerates deterministically.

- [ ] **PS504 `HWConfirmTariffChangeAction` → ?**

  FIX Yours to close: make the query static, or add WITH USER_MODE so the runtime enforces this user's access whatever the query resolves to.

- [ ] **PS504 `HWProposeTariffChangeAction` → ?**

  FIX Yours to close: make the query static, or add WITH USER_MODE so the runtime enforces this user's access whatever the query resolves to.

- [ ] **PS514 `HWConfirmTariffChangeAction (platform event: Tariff_Change_Requested__e)`**

  FIX List the subscribers of Tariff_Change_Requested__e (Flow, process, and external) and review each as its own entry point.

FINDINGS IN DETAIL

WARN · 3

PS504 `HWConfirmTariffChangeAction → ?`

Reach for this operation could not be fully determined - the query is assembled at runtime (dynamic SOQL - reach cannot be determined statically).

Why A silent false-clean is worse than an honest unknown, so this is counted as unresolved rather than passed. The query does not exist until runtime, so no static analysis - this one or any other - can enumerate what it reaches. It stays unresolved until the code changes.

Fix Yours to close: make the query static, or add WITH USER_MODE so the runtime enforces this user's access whatever the query resolves to.

PS504 `HWProposeTariffChangeAction → ?`

Reach for this operation could not be fully determined - the query is assembled at runtime (dynamic SOQL - reach cannot be determined statically).

Why A silent false-clean is worse than an honest unknown, so this is counted as unresolved rather than passed. The query does not exist until runtime, so no static analysis - this one or any other - can enumerate what it reaches. It stays unresolved until the code changes.

Fix Yours to close: make the query static, or add WITH USER_MODE so the runtime enforces this user's access whatever the query resolves to.

PS514 `HWConfirmTariffChangeAction (platform event: Tariff_Change_Requested__e)`

This action publishes Tariff_Change_Requested__e. The publish is analysed as a write; any Flow, process, or external subscriber is NOT.

Why No Apex subscriber trigger exists on it, but a platform event can also be consumed by a Flow, a process, or an off-platform subscriber, each running in its own transaction as a different user. Publishing is how an agent starts work it could not do inline, so the true blast radius can be larger than this report. An honest unknown edge, not a proven leak.

Fix List the subscribers of Tariff_Change_Requested__e (Flow, process, and external) and review each as its own entry point.

ORG HEALTH — BEYOND THIS AGENT

Whole-org signals that don't concern **HW Energy Agent** directly, but anyone securing this org should know. Static Tooling-API reads — zero credits, no records returned.

> **Why this matters for HW Energy Agent:** an agent can only reach what the code behind it reaches. Where no explicit access mode overrides it, the Apex **API version** sets the default for every database operation. At **v67+** that default is the running user's mode, so an agent reaches *nothing its user cannot see*. **Below v67** it flips to system mode and the agent reaches *past* its user. An explicit `WITH SYSTEM_MODE` still overrides either. HW Energy Agent has **0 proven** escalation — but 2 operations could not be resolved at all, so this is **not** a clean result: an unknown never becomes clean. And if the code is still pre-v67, even that zero rests on the code *explicitly* opting in (`WITH USER_MODE` / `as user`), not on the platform default.
>
> The whole-org counts below are that same condition at scale — a map of where the *next* agent or action will escalate, before it is built.

182/219	1	3	0
YOUR APEX STILL PRE-V67 83% system-mode by default	GRANT "MODIFY ALL DATA"	GRANT "VIEW ALL DATA" ONLY	PUBLIC-OWD CUSTOM OBJECTS

Why this matters: a class or trigger below API v67 defaults to **system mode**, so its queries and writes bypass the running user's sharing and field-level security unless the code opts in — this agent uses only a couple of them, but the same latent escalation sits in every other pre-v67 file; **1** permission set grant **Modify All Data**, which overrides all sharing and FLS for whoever holds it — an agent running as such a user has no access boundary at all; every custom object defaults to **Private** sharing — a healthy baseline.

Produced by static analysis. No agent was invoked. 0 Flex Credits. Bound to fingerprint da5d75d3171b, which seals both the INPUTS (agent config, the analysed Apex/Flow, the permission snapshot, and what the analysis identity could see) and the TOOL that produced this verdict (analyzer 52c669ea07a9, parser 5.1.0; each analysed class's own apiVersion is bound per action, since it decides the verdict). Regenerate if any of these change. The Aksu Index is defined by a public specification, v1.0: aksuindex.com